

# **Beyond Representation: Index Structure Limitations in Dense Retrieval**

Alessandro Magnani

Advanced Machine Learning & Data Science  
AMLDS 2026 | Osaka, Japan

July 22, 2026

# Why this talk: can we drop the sparse side?

**Production retrieval today is hybrid:** a dense embedding index (HNSW / IVF) *plus* an inverted index (BM25). Two stores, two scoring paths, extra ops.

**The tempting question:** could a pure-dense stack replace hybrid? Simpler, one index, one score.

Two things have to hold:

- ▶ **Bottleneck 1 — representation.** Can the embedding *encode* what BM25 encodes? Well-studied: Tay et al. 2020, Fan et al. 2022 — capacity grows with  $d$ .
- ▶ **Bottleneck 2 — indexing.** Even if the embedding is capable, can an *ANN index* still find it? **This talk. Largely unstudied.**

## Headline (the index alone — same embedding)

**Flat** → **IVF** (partition index the theory targets): forfeits  $\sim 33\%$  of achievable Recall@10 on identifier queries,  $< 5\%$  on standard.

**Flat** → **HNSW** (graph index, same population):  $\sim 63\%$  on identifier queries,  $< 2\%$  on standard.

# The population that pays the bill: “spear-fishing” queries

## Working definition

Queries whose relevance hinges on a single, low-frequency, high-IDF token: identifiers (SKUs, part numbers), rare terms, precise named entities.

- ▶ Small fraction of traffic, but operationally critical: failed spear-fishing  $\Rightarrow$  empty result pages, abandoned carts, broken RAG lookups.
- ▶ BM25 handles them trivially (exact token match).
- ▶ If pure dense is going to replace hybrid, it has to hold up *here*.
- ▶ Our three query subsets: **standard** (control), **rare** ( $\leq 5$  docs), **ID** (extreme case: doc identifier as query).

# The idealized case: BM25 is embedding search

**Reframe BM25 as a dense inner product** in a huge, discrete embedding space  $\mathbb{R}^V$  (one dimension per token,  $V \approx 10^5$ ).

- ▶ Each doc  $d$  is a sparse vector  $f(d) \in \mathbb{R}^V$  with  $f(d)_t = \alpha_t$  (IDF weight) if  $t \in d$ , else 0.
- ▶ Query  $q$  dominated by rare token  $t^*$ :  $q$  has weight  $\alpha_{t^*}$  in that direction.
- ▶ Inner product with a relevant doc  $d_{\text{rel}}$  (contains  $t^*$ ):

$$\langle q, f(d_{\text{rel}}) \rangle = \alpha_{t^*}^2 \quad (\text{big} \text{ --- } \alpha_{t^*} = \log(M/n_{t^*})).$$

- ▶ Inner product with an irrelevant doc: essentially 0.
- ▶ **Perfect separation on spear-fishing queries.** This is exactly why BM25 works. It is a nearest-neighbor search in  $\mathbb{R}^V$ .

**The catch:**  $V$  is huge. Storing and searching  $\mathbb{R}^V$  is what BM25 does with an inverted list — practical, but the wrong shape for hybrid or pure dense. To use ANN we must compress  $\mathbb{R}^V \rightarrow \mathbb{R}^d$  with  $d \ll V$ .

*The rest of this talk is what compression costs.*

# Compressing $\mathbb{R}^V \rightarrow \mathbb{R}^d$ : Johnson–Lindenstrauss

Random linear map  $A : \mathbb{R}^V \rightarrow \mathbb{R}^d$ ,  $d \in \{256, 512, 1024\}$ .

## JL theorem (informal)

Given  $M$  vectors in  $\mathbb{R}^V$ , a random projection  $A$  preserves all pairwise inner products *up to a multiplicative error*  $\epsilon$  with high probability, provided

$$d \gtrsim \frac{\log M}{\epsilon^2} \iff \epsilon \approx \sqrt{\frac{\log V}{d}}.$$

## What survives, what dies:

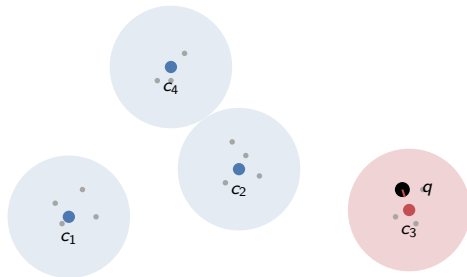
- ▶ Any inner product with magnitude  $\gg \epsilon$  is preserved.  $\Rightarrow$  Spear-fishing signal  $\alpha_{t^*}^2$  from the previous slide easily clears  $\epsilon$ : **Flat search on the projected embedding is a clean oracle.**
- ▶ Any inner product with magnitude  $\lesssim \epsilon$  is drowned in projection noise — indistinguishable from random.
- ▶ Bigger  $d$  shrinks  $\epsilon$ ; nothing else does.

So after JL alone, spear-fishing still works. The failure emerges when we add the *second* compression — clustering — on top. That is IVF.

## Quick primer: what is an IVF index?

**IVF** (Inverted File) — the standard partition-based ANN index.

1. Cluster the corpus into  $N$  cells (k-means). Each cell has a *centroid*  $c$ .
2. At query time: score the query against all  $N$  centroids; keep the top  $n_{\text{probe}}$ .
3. Only search documents inside those  $n_{\text{probe}}$  cells.



**Speedup:**  $N/n_{\text{probe}}$  fewer distance computations. **Failure mode:** if the relevant cell's centroid is not in the top  $n_{\text{probe}}$  by query similarity, its documents are *invisible*.

# The mechanism: centroid dilution (intuition)

A centroid is the *average* of the vectors in its cell.

- ▶ A *common* token appears in many docs  $\Rightarrow$  contributes strongly to the average  $\Rightarrow$  shows up in the centroid.
- ▶ A *rare* token appears in a handful of docs  $\Rightarrow$  its contribution is averaged away  $\Rightarrow$  nearly invisible in the centroid.

Now consider a query dominated by that rare token:

- ▶ Query vector has huge weight on the rare token direction.
- ▶ Every centroid — *including the one containing the relevant docs* — has almost zero weight on that direction.
- ▶ Result: **all centroids look equally (un)related to the query**. Cluster selection becomes essentially random.

*The failure is not the embedding. It is the averaging step in the index.*

# The bound: when does the centroid signal survive?

Recall: in  $\mathbb{R}^V$ ,  $\langle q, f(d_{\text{rel}}) \rangle = \alpha_{t^*}^2$ ; JL preserves it. IVF selects clusters by **centroid** inner product  $\Rightarrow$  average kicks in.

## Notation

- ▶  $M$  = corpus size (# docs)
- ▶  $N$  = # IVF clusters
- ▶  $C$  = one cluster,  $|C| \approx M/N$
- ▶  $t^*$  = rare token dominating the query
- ▶  $n_{t^*}$  = # docs in corpus with  $t^*$
- ▶  $n_{t^*,C}$  = # docs in  $C$  with  $t^*$
- ▶  $\alpha_{t^*} = \log(M/n_{t^*})$  (IDF)
- ▶  $\epsilon$  = JL noise floor

**Step 1.** Centroid = average of docs in the cluster:

$$c_C = \frac{1}{|C|} \sum_{d \in C} f(d).$$

**Step 2.**  $t^*$  contributes to  $c_C$  only through the  $n_{t^*,C}$  docs in  $C$  that contain it:

$$c_C|_{t^*} \approx \underbrace{\frac{n_{t^*,C}}{|C|}}_{p = \text{prevalence}} \cdot \alpha_{t^*}.$$

Rare token  $\Rightarrow p$  tiny.

**Step 3.** Query dominated by  $t^*$  (weight  $\alpha_{t^*}$ ):

$$\langle c_C, q \rangle \approx \alpha_{t^*}^2 \cdot p$$

## Survival condition

Relevant centroid recognizable iff  $\alpha_{t^*}^2 \cdot p \gg \epsilon$ . Rare:  $\alpha$  big,  $p$  tiny  $\Rightarrow$  product  $< \epsilon \Rightarrow$  wrong cluster picked.  
*Bigger  $d$  helps  $\epsilon$ ; nothing helps  $p$ .*

# HNSW isn't a way out

**HNSW** is not partition-based — it is a *layered proximity graph*. Greedy descent from a hub node down to the query's local neighborhood.

## So why does it fail on the same queries?

- ▶ Graph edges are built from Euclidean distances during insertion.
- ▶ Dense regions of the embedding space get thick, well-connected subgraphs. Rare vectors land in sparse regions — *few incoming edges from any hub*.
- ▶ Greedy search from a hub cannot reach an orphan node without a long detour: it stops at a locally-good but globally-wrong region.
- ▶ Widening the beam (efSearch) helps — but with diminishing returns.

**The pattern is the same:** rare-token vectors are structurally hard to *reach*, whether the structure is a partition (IVF) or a graph (HNSW). Different mechanism; **same victims**.

# Experimental setup (compact)

## Datasets (1,000 queries per subset)

- ▶ Home Depot (HD): 54,667 product titles
- ▶ MS MARCO Passage v1.1: 82,360 passages

## Query subsets

- ▶ standard | rare ( $\leq 5$  docs) | ID

**Representation:** char-trigram + JL projection,  $d \in \{256, 512, 1024\}$ , unsupervised.

**Indices (FAISS):** Flat, IVF, HNSW ( $M=16$ , efC=200), BM25 baseline (Pyserini).

## Why unsupervised trigrams?

A supervised encoder abstracts away the lexical detail that *defines* spear-fishing queries. To isolate the *index* bottleneck from the representation bottleneck we need a representation that provably preserves lexical distances (Johnson–Lindenstrauss).

## Headline result: the Flat $\rightarrow$ HNSW gap

| Dataset  | Subset    | Flat         | IVF   | HNSW         | BM25  |
|----------|-----------|--------------|-------|--------------|-------|
| HD       | standard  | 0.327        | 0.314 | 0.321        | 0.447 |
| HD       | rare      | 0.751        | 0.642 | 0.619        | 0.878 |
| HD       | <b>id</b> | <b>0.874</b> | 0.590 | <b>0.327</b> | 1.000 |
| MS MARCO | standard  | 0.471        | 0.448 | 0.446        | 0.770 |
| MS MARCO | rare      | 0.205        | 0.187 | 0.166        | 0.688 |
| MS MARCO | <b>id</b> | <b>0.174</b> | 0.152 | <b>0.071</b> | —     |

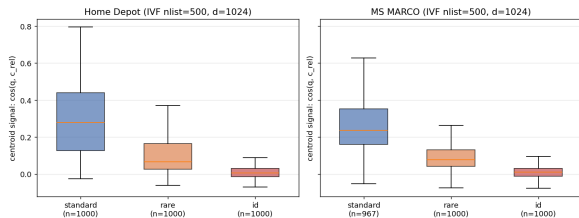
Recall@10 at  $d=1024$ , best ANN sweep config.

- ▶ Relative Flat $\rightarrow$ HNSW loss: HD {standard **1.8%**, rare **17.5%**, **id 62.6%**}; MM analogous.
- ▶ The gap tracks the query population exactly as the theory predicts.

# Direct evidence: measure the centroid signal

**For each query:** compute  $\langle q, c_{\text{rel}} \rangle$  on the centroid of the query's *relevant* cluster (IVF,  $n_{\text{list}}=500$ ,  $d=1024$ , HD).

Centroid signal for relevant cluster, by query subset

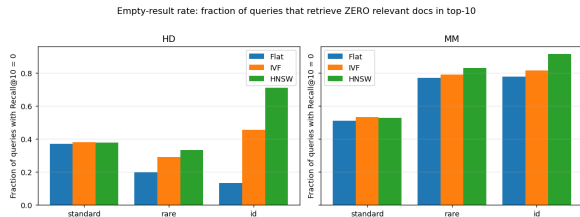


| Subset   | signal | rank | hit@250 |
|----------|--------|------|---------|
| standard | 0.279  | 2    | 97%     |
| rare     | 0.066  | 69   | 81%     |
| id       | 0.007  | 216  | 59%     |

~ 40× collapse of the centroid signal from standard to ID — the correct cluster is buried at rank ~ 200.

The bound  $\langle c, q \rangle \approx \alpha_{t^*}^2 p$  is not just suggestive — it is what we *measure*.

# The production-visible symptom



## Fraction of queries with zero relevant docs in top-10:

- ▶ HD-id: Flat **13%** → HNSW **71%**
- ▶ MM-id: Flat **78%** → HNSW **92%**
- ▶ Standard: within  $\pm 2$  pp on both datasets.

## Cost to match Flat recall (efSearch, HD):

- ▶ standard: **2.7 $\times$**  faster than Flat
- ▶ id: **40 $\times$**  slower than Flat

ANN's speedup evaporates on the population that needs it most.

## “But learned encoders would fix this”

Anticipated question: *do supervised dense encoders escape the issue?*

Re-ran with all-MiniLM-L6-v2 (Sentence-BERT, 384-dim):

| Dataset  | Subset   | Trigram Flat R@10 | MiniLM Flat R@10 |
|----------|----------|-------------------|------------------|
| HD       | standard | 0.327             | 0.298            |
| HD       | rare     | 0.751             | 0.428            |
| HD       | id       | 0.874             | 0.010            |
| MS MARCO | standard | 0.471             | 0.894            |
| MS MARCO | id       | 0.174             | 0.001            |

- ▶ MiniLM *cannot distinguish identifiers* — “12345”  $\approx$  “67890”.
- ▶ Representation collapses *before* the index has a chance to bias.
- ▶ Which is precisely why hybrid retrieval exists.

# Takeaways

1. Production dense retrieval is hybrid for a reason. If you want to drop the sparse side, you have to clear *two* bottlenecks, not one.
2. Bottleneck 1 (representation) is well-studied. Bottleneck 2 (indexing) is what this paper isolates and quantifies.
3. Centroid dilution + JL noise floor give a closed-form bound; the  $\sim 40\times$  centroid-signal collapse from standard to ID is exactly what the bound predicts.
4. HNSW is not a workaround — the mechanism differs, the victims don't.
5. Practical implication: query-type-aware routing, index-aware training, or — most simply — keep the sparse side.

**Code & experiments:** [github.com/alemagnani/sparse-dense-paper](https://github.com/alemagnani/sparse-dense-paper)

# Thank you

Questions?

Alessandro Magnani | [ale.magnani@gmail.com](mailto:ale.magnani@gmail.com)  
AMLDS 2026 | Osaka, Japan | July 22, 2026